TD supplémentaire SQL

Révision SQL (LMD et LDD)

2026-11-06

Avec solutions

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle

Objectifs

- révision des requêtes de manipulation de données (LMD) en SQL,
- révision des requêtes de définition de données (LDD) en SQL.

Le TD utilise le schéma `pagila` et votre schéma personnel. Pensez à adapter le `search_path` de votre base de données pour alléger le code !

Questions

1. Afficher les acteurs (identifiant,nom et prénom) dont le nom de famille se termine par la lettre "a" (peu importe la casse). Trier les résultats par nom de famille dans l'ordre alphabétique. *3 lignes attendues.*

Solution

```
select actor_id,last_name,first_name
from actor
where last_name ilike '%a'
order by last_name asc;
```

2. Afficher les films (identifiant, titre, tarif de location) dont le tarif de location ('rental_rate') est strictement inférieur à 3. Trier les résultats par tarif de location décroissant. *664 lignes attendues.*

Solution

```
select film_id, title, rental_rate
from film
where rental_rate < 3
order by rental_rate desc;
```

3. Afficher la liste sans doublons de tous les districts présents dans la table "address". Trier les districts par ordre alphabétique. *378 lignes attendues.*

Solution

```
select distinct district
from address
order by district asc;
```

4. Afficher les films (identifiant, titre, description) dont la description contient le mot “dog” ou le mot “cat”, insensible à la casse. *167 lignes attendues.*

Solution

```
select film_id, title, description
from film
where (description ilike '%dog%') or (description ilike '%cat%')
order by title asc;
```

5. Afficher la liste de tous les acteurs avec tous les films dans lesquels ils ont joué. Il faut afficher les colonnes identifiant d’acteur, prénom, nom, identifiant du film et titre du film. Trier les résultats par nom puis prénom d’acteur puis par titre de film. *5462 lignes attendues.*

Solution

```
select a.actor_id, a.first_name, a.last_name, f.film_id, f.title
from actor a left join film_actor fa using (actor_id)
   join film f using (film_id)
order by a.last_name asc, a.first_name asc, f.title asc;
```

6. Afficher sans doublons les films (identifiant et titre), dans lesquels joue au moins une actrice ou un acteur dont le nom commence par la lettre ‘B’ et au moins une actrice ou un acteur dont le nom commence par la lettre ‘M’. Trier les résultats par titre de film. *172 lignes attendues.*

Solution

Solution sans CTEs :

```
select distinct f.film_id, f.title
from film f join film_actor fa1 on f.film_id = fa1.film_id
   join actor a1  on fa1.actor_id = a1.actor_id
   join film_actor fa2  on f.film_id = fa2.film_id
   join actor a2  on fa2.actor_id = a2.actor_id
where a1.last_name like 'B%' and a2.last_name like 'M%'
order by f.title asc;
```

Avec CTEs :

```
with film_b as (
    select distinct f.film_id, f.title
    from film f
    join film_actor using (film_id)
    join actor a using (actor_id)
    where a.last_name like 'B%'
),
film_m as (
    select distinct f.film_id, f.title
    from (film f
    join film_actor using (film_id))
    join actor a using (actor_id)
    where a.last_name like 'M%'
)
select  b.film_id, b.title
from film_b b join film_m using (film_id)
order by b.title;
```

Avec `intersect` (`distinct` inutile car l'intersection supprime les doublons) :

```
with filmbm as (
    (
        select f.film_id, f.title
        from film f
        join film_actor using (film_id)
        join actor a using (actor_id)
        where a.last_name like 'B%'
    )
    intersect
    (
        select f.film_id, f.title
        from film f
        join film_actor using (film_id)
        join actor a using (actor_id)
        where a.last_name like 'M%'
    )
)
select *
from filmbm
order by title;
```

La requête suivante ne répond pas à la question car elle sélectionne les films dans lesquels jouent des acteurs dont le nom commence par 'B' **ou** dont le nom commence par 'M'. On obtient donc une réunion d'ensembles, là où on veut une intersection.

```
select f.film_id, f.title, a.last_name
from film f
   join film_actor fa on f.film_id = fa.film_id
   join actor a on fa.actor_id = a.actor_id
where a.last_name like 'B%' or a.last_name like 'M%'
order by f.title;
```

7. Afficher le coût de remplacement (‘replacement_cost’) minimum, maximum et moyen de l’ensemble des films. *Valeurs attendues : minimum = 9.99, maximum = 29.99 et moyen = 19,984.*

Solution

```
select min(replacement_cost) as cout_remplacement_minimum,
       max(replacement_cost) as cout_remplacement_maximum,
       avg(replacement_cost) as cout_remplacement_moyen
from film;
```

8. Afficher, pour chaque client, la dépense totale théorique basée sur la somme du montant (‘amount’) de la table “payment” des films loués. Trier du plus gros total au plus faible. Les colonnes à afficher sont l’identifiant du client, le prénom, le nom et la dépense totale théorique. *599 lignes attendues, la première ligne est : 526 KARL SEAL 221,55.*

Solution

```
with custopay as (
    select customer_id, sum(amount) as depense
    from payment
    group by customer_id
)
select customer_id, first_name, last_name, depense
from custopay join customer using (customer_id)
order by depense desc;
```

9. Afficher toutes les locations dont la durée (‘return_date’ - ‘rental_date’) est supérieure à la durée moyenne de toutes les locations. Trier les résultats par durée décroissante. Les colonnes à afficher sont : ‘rental_id’, ‘customer_id’, ‘duree_location’ (alias). *7923 lignes attendues, la première ligne est : 2412 127 9 days 05 :59 :00.*

Solution

Avec jointure, sans requête imbriquée :

```
with moyrent as (
    select avg(return_date - rental_date) as moy
    from rental
)
select rental_id, customer_id, return_date - rental_date as duree_location
from rental, moyrent
where return_date - rental_date > moy
order by duree_location desc;
```

Avec une requête imbriquée :

```
with moyrent as (
    select avg(return_date - rental_date) as moy
    from rental
)
select rental_id, customer_id, return_date - rental_date as duree_location
from rental
where return_date - rental_date > (
      select *
      from moyrent
   )
order by duree_location;
```

10. Afficher les films de la catégorie “Horror” dont la durée (‘length’) est inférieure à la durée moyenne des films d’horreur. Les colonnes à afficher sont l’identifiant du film, le titre et la

durée. Trier les résultats par durée décroissante. *28 lignes attendues, la première ligne est : 965 WATERSHIP FRONTIER 112.*

Solution

Solution 1 avec jointure et CTEs :

```
-- cherchons les films d'horreurs, et gardons leur identifiant, titre et durée
with filmhor as (
    select f.film_id as id,f.title as titre , f.length as duree
    from film f
    join film_category using (film_id)
    join category c using (category_id)
    where c.name like 'Horror'
),
-- calculons la durée moyenne
durhor as (
    select avg(duree) as moy from filmhor
)
--maintenant la requete
select id, titre, duree
from filmhor as f , durhor as d
where f.duree < d.moy
order by duree desc;
```

Solution 2 avec une requête imbriquée sans CTEs :

```
select f.film_id, f.title, f.length
from film as f join film_category as fc
on f.film_id=fc.film_id
join category as c
on c.category_id=fc.category_id
where c.name like 'Horror'
    and f.length < (
        select avg(f.length)
        from film as f join film_category as fc
                on f.film_id=fc.film_id
            join category as c
                on c.category_id=fc.category_id
        where c.name like 'Horror'
    )
order by f.length desc;
```

Comparatif :

- La solution 1 est plus lisible et plus facile à comprendre. La solution 2 est plus compacte mais moins lisible.
- La solution 1 est plus efficace car elle ne calcule la durée moyenne qu'une seule fois, alors que la solution 2 la calcule pour chaque film d'horreur.

11. Dans votre schéma personnel, créer une table `td_film` de structure identique à la table "film" du schéma "pagila". Les contraintes internes aux deux tables doivent être identiques.

Solution

```
create table if not exists username.td_film (like pagila.film including all);
```

12. Insérer dans la table `td_film` de votre schéma les films de la table `pagila.film` dans lesquels jouent au moins une actrice ou un acteur dont le nom commence par la lettre 'B'. *449 lignes insérées attendues.*

Solution

Sans CTE (`distinct` nécessaire car un film peut avoir plusieurs acteurs dont le nom commence par 'B', donc on risque d'avoir des doublons interdits par la contrainte de clé primaire) :

```
insert into username.td_film
(
    select distinct f.*
    from pagila.film as f
      join pagila.film_actor as fa on f.film_id=fa.film_id
        join pagila.actor as a on a.actor_id=fa.actor_id
    where a.last_name like 'B%')
;
```

Avec CTE (idem pour le `distinct`) :

```
with film_a_inserer as (
    select f.film_id
    from pagila.film f
       join pagila.film_actor using (film_id)
       join pagila.actor a using (actor_id)
    where a.last_name like 'B%'
)
insert into username.td_film (
    select distinct *
    from pagila.film join film_a_inserer using (film_id)
);
```

13. Sans supprimer les données existantes dans votre table `td_film` et sans provoquer d'erreur d'exécution, insérer dans votre table `td_film` les films de la table `pagila.film` dans lesquels jouent au moins une actrice ou un acteur dont le nom commence par la lettre 'M'. *168 lignes insérées attendues.*

Solution

En insérant tous les films dont le nom d'acteur commence par 'M', on risque d'avoir des doublons avec les films déjà insérés dans l'exercice précédent (acteurs dont le nom commence par 'B').
Solution standard : on peut utiliser une requête imbriquée pour ne sélectionner que les films dont l'identifiant n'est pas déjà présent dans la table `td_film` de notre schéma. Cela permet d'éviter les doublons et donc les erreurs d'insertion. `not exists` est plus efficace car il s'arrête dès qu'il trouve un match, contrairement à `not in` qui doit construire la liste complète des valeurs.

```
with film_a_inserer as (
    select f.film_id
    from pagila.film f
      join pagila.film_actor using (film_id)
      join pagila.actor a using (actor_id)
    where a.last_name like 'M%'
)
insert into username.td_film (
    select distinct *
    from pagila.film f1 join film_a_inserer using (film_id)
    where not exists (
      select 1
      from username.td_film f2
      where f2.film_id = f1.film_id
   )
);
```

Ou bien avec `not in`, moins efficace (attention aussi aux `null` dans une sous-requête avec `not in`, mais ici `film_id` est une clé primaire donc pas de problème) :

```
with film_a_inserer as (
    select f.film_id
    from pagila.film f
      join pagila.film_actor using (film_id)
      join pagila.actor a using (actor_id)
    where a.last_name like 'M%'
)
insert into username.td_film (
    select distinct *
    from pagila.film f1 join film_a_inserer using (film_id)
    where f1.film_id not in (
      select film_id
      from username.td_film
   )
);
```

Solution non standart : le plus simple avec PostgreSQL est d'utiliser `insert ... on conflict do nothing` pour ignorer les doublons. Cette clause est disponible en PostgreSQL depuis la version 14. **Mais comme le standard SQL n'autorise pas cette clause, elle n'est pas présente dans les autres SGBD** : MySQL, Oracle, SQL Server, etc. Le code suivant n'assure donc pas la portabilité du code SQL, ce qui est un **inconvénient majeur** pour l'évolution future de la base de données. Pour différentes raisons, l'organisation qui gère le base de données pourrait décider à l'avenir de migrer vers un autre SGBD. Devoir réécrire toutes les requêtes SQL pour les rendre compatibles avec le nouveau SGBD serait très coûteux. Il est donc préférable d'utiliser des requêtes SQL standard, même si elles sont plus complexes à écrire. Pour plus de détails sur la clause `on conflict do nothing`, voir la documentation PostgreSQL.

```
insert into username.td_film (
    select f.* from pagila.film as f
    where f.film_id in (
        select distinct fa.film_id from pagila.film_actor as fa
        join pagila.actor as a on a.actor_id = fa.actor_id
```